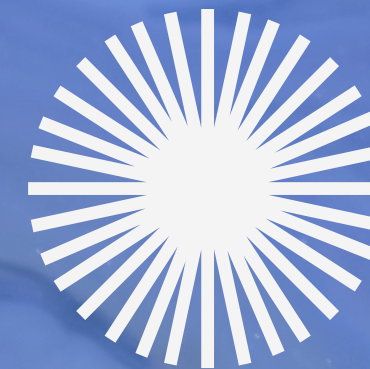


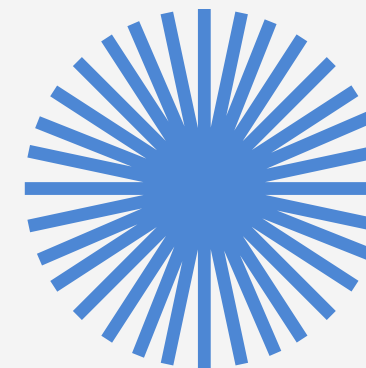


T1 – Testare unitară în Python

➤ Echipa 18
➤ Dragomir Miruna, Iștoc Simona, Tanislav Alexia, Zamfir Alexandru

➤ 18.05.2026

TaxVision RO



01

Scopul Aplicației ↗

02

Tehnologii &
Decizii de Proiectare ↗

03

Arhitectura
Proiectului ↗

04

Strategii de Testare ↗

05

Testarea
prin Mutații ↗

TaxVision RO ajută utilizatorii să analizeze impactul fiscal asupra veniturilor din România printr-un flux de lucru comparativ.

Obiectiv Principal

Demonstrarea corectitudinii algoritmilor fiscali prin strategii de testare riguroase și analiză de acoperire a codului (Code Coverage).



TESTAREA SISTEMELOR SOFTWARE

Scopul Aplicației

01.

Compararea Simulărilor

Istoric complet al simulărilor de venit pentru urmărirea și revizuirea lor ulterioară.



02.

Export Date

Exportul rezultatelor în format Excel și CSV pentru raportare și analiză suplimentară.



03.

Vizualizare grafică

Reprezentări grafice ale rezultatelor pentru interpretarea mai ușoară a diferențelor.



01. pytest VS unittest

Am ales **pytest** deoarece:

- este mai ușor de scris și citit
- organizare pe strategie de testare
- mai puțin boilerplate față de unittest
- integrare naturală cu mutation testing
- monkeypatch disponibil nativ

02. Streamlit VS Django

Am ales **Streamlit** pentru că:

- focus pe logica fiscală, nu pe infrastructură web
- iterare rapidă a interfeței
- fără modele, routing, templates extra
- inputs, grafice și export cu cod minim
- proiect lightweight, ușor de testat

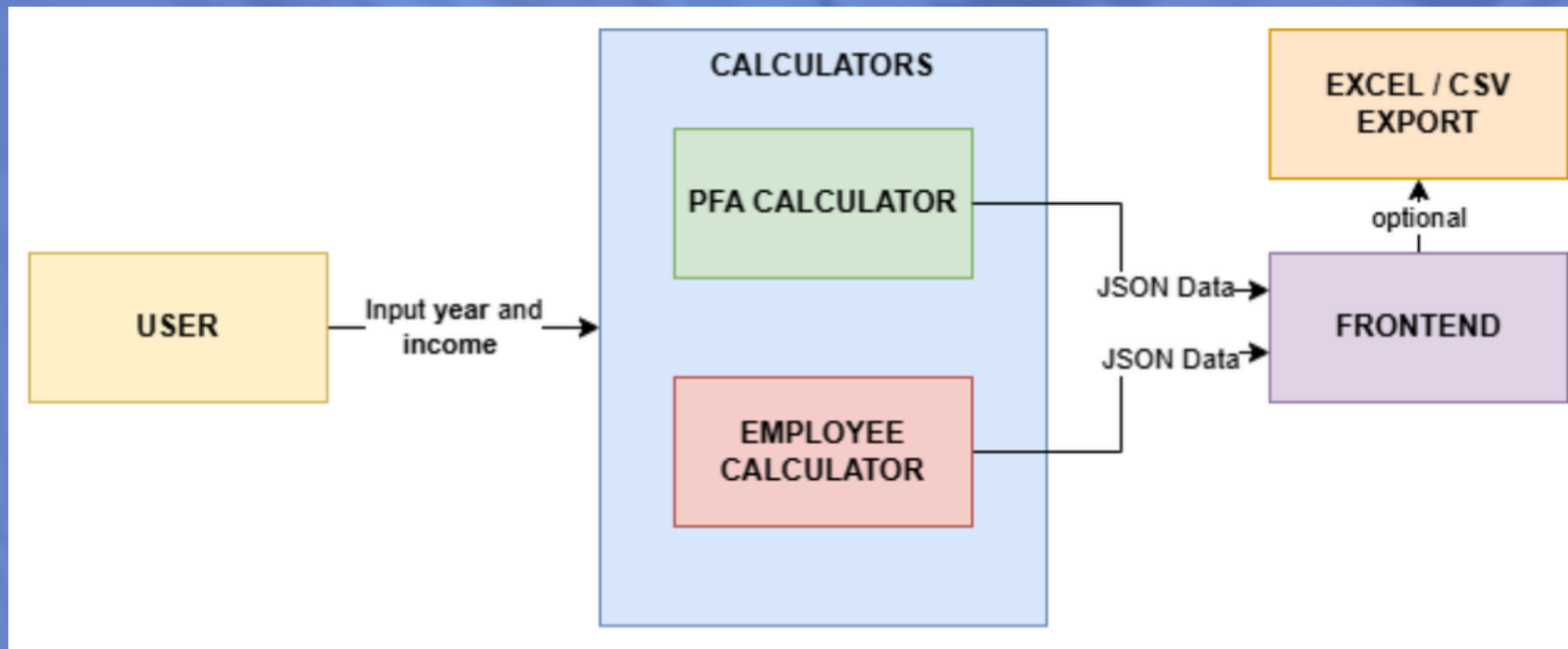
03. cosmic-ray VS mutmut

Am ales **cosmic-ray** pentru că:

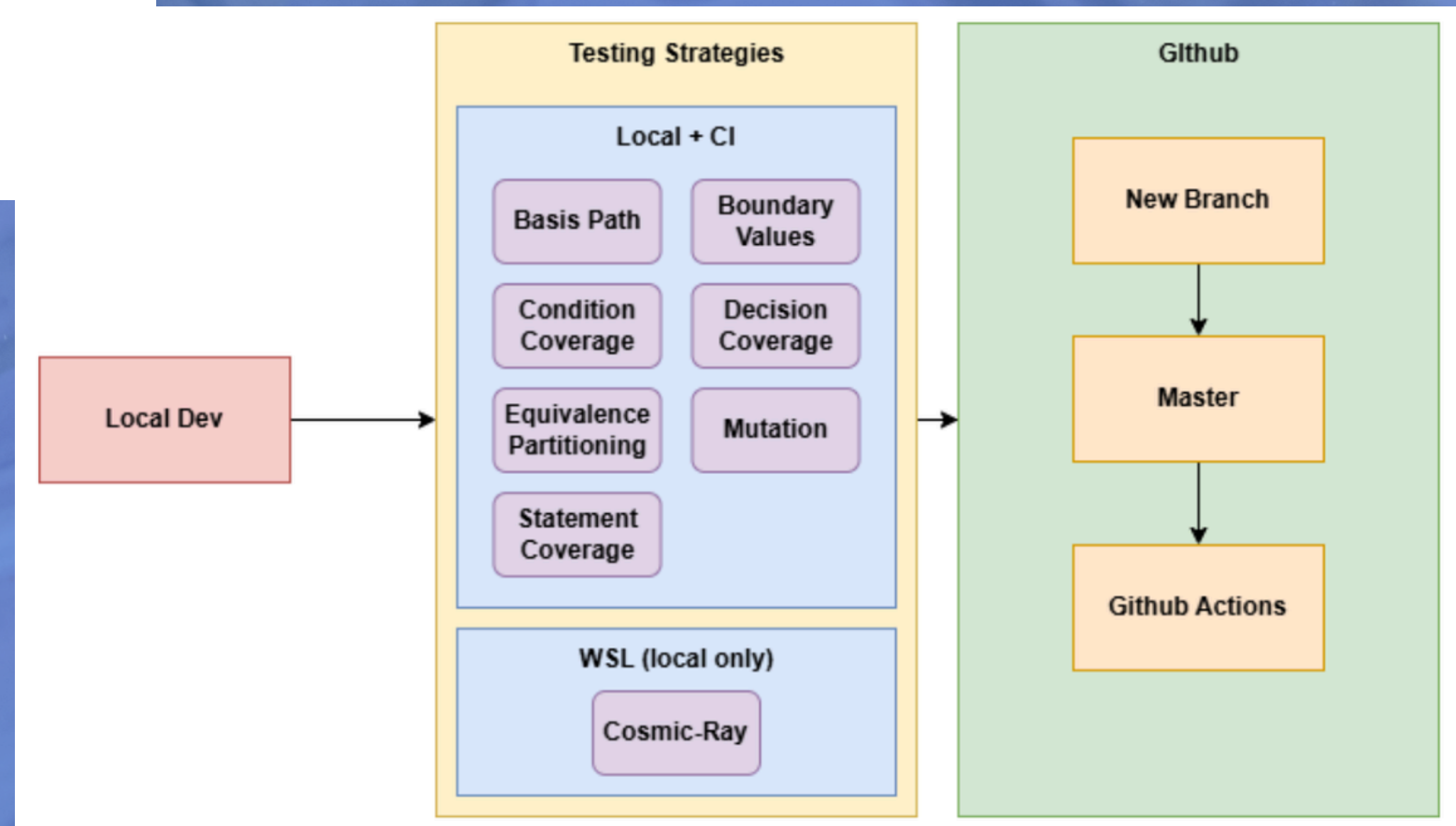
- este o alternativă potrivită pentru mutmut, care nu suportă Windows nativ
- are același flux: init → exec → cr-html pentru raport vizual

Tehnologii & Decizii de Proiectare

Application Flow



Development & Testing PIPELINE

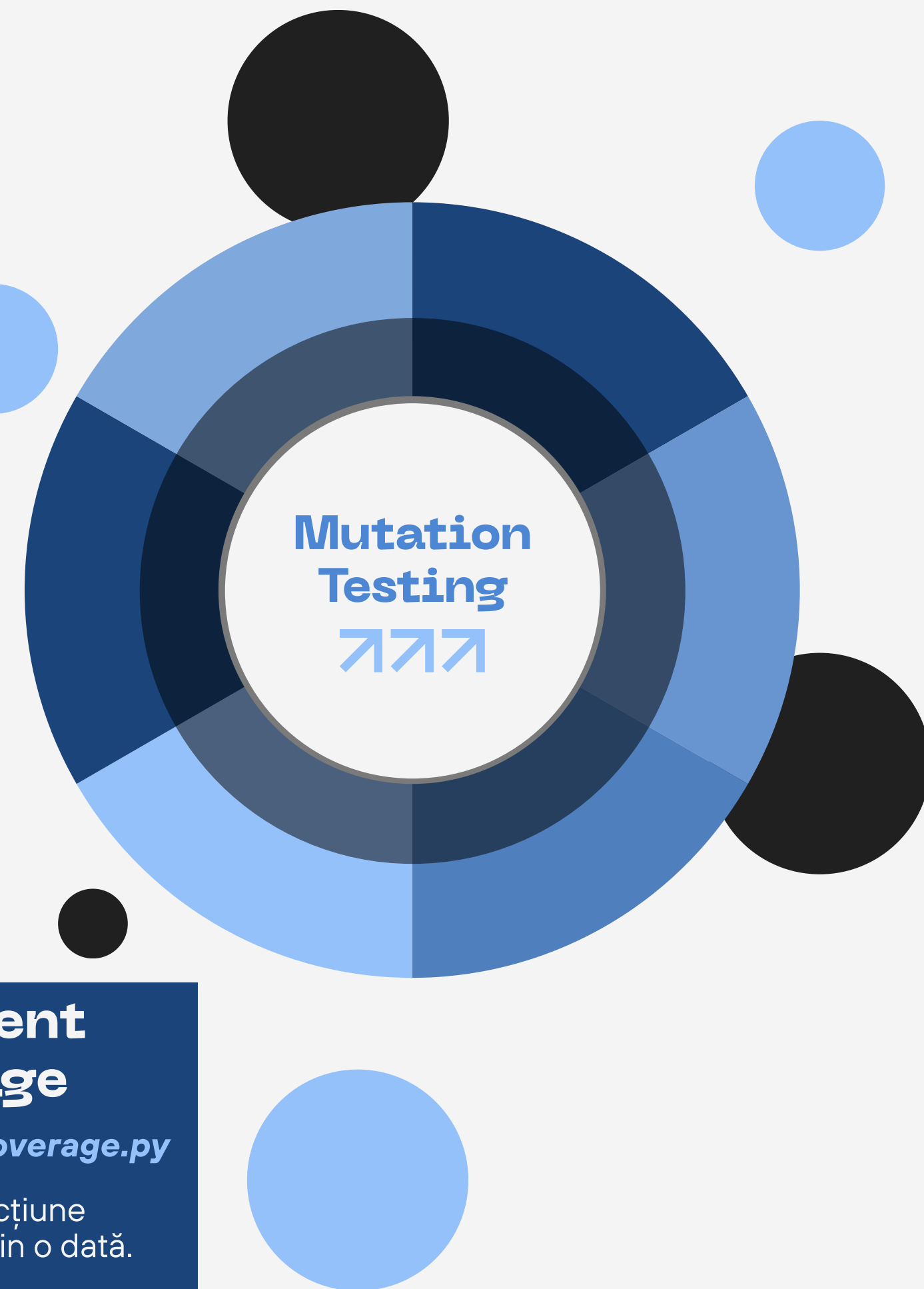
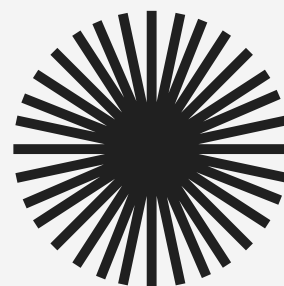


Arhitectura Proiectului

Logică de calcul (/src)	
calculator_base.py	clasă abstractă de bază
calculator_employee.py	calcul salariat
calculator_pfa.py	calcul PFA
ax_config.py	configurare fiscală
tax_rules.json	reguli pe ani fiscali
app.py	interfață Streamlit
app_helpers.py	logică UI auxiliară

Suită de teste (/tests)
test_statement_coverage.py
test_decision_coverage.py
test_condition_coverage.py
test_boundary_values.py
test_equivalence_partitioning.py
test_basis_path.py
test_mutation_analysis.py
test_app_helpers_run_simulation.py

Strategii de Testare



Basis Path

test_basis_path.py

Căile sunt extrase pe baza CFG-urilor, fiecare test adăugând o muchie nouă.

Boundary Value

test_boundary_values.py

Limite pentru venit (0, -0.01, 0.01) și praguri CAS/CASS.

Condition Coverage

test_condition_coverage.py

Fiecare condiție individuală evaluată True și False.

Decision Coverage

test_decision_coverage.py

Fiecare ramură if/else acoperită.

Equivalence partitioning

test_equivalence_partitioning.py

Clase valide și invalide pentru venit și an fiscal.

Statement Coverage

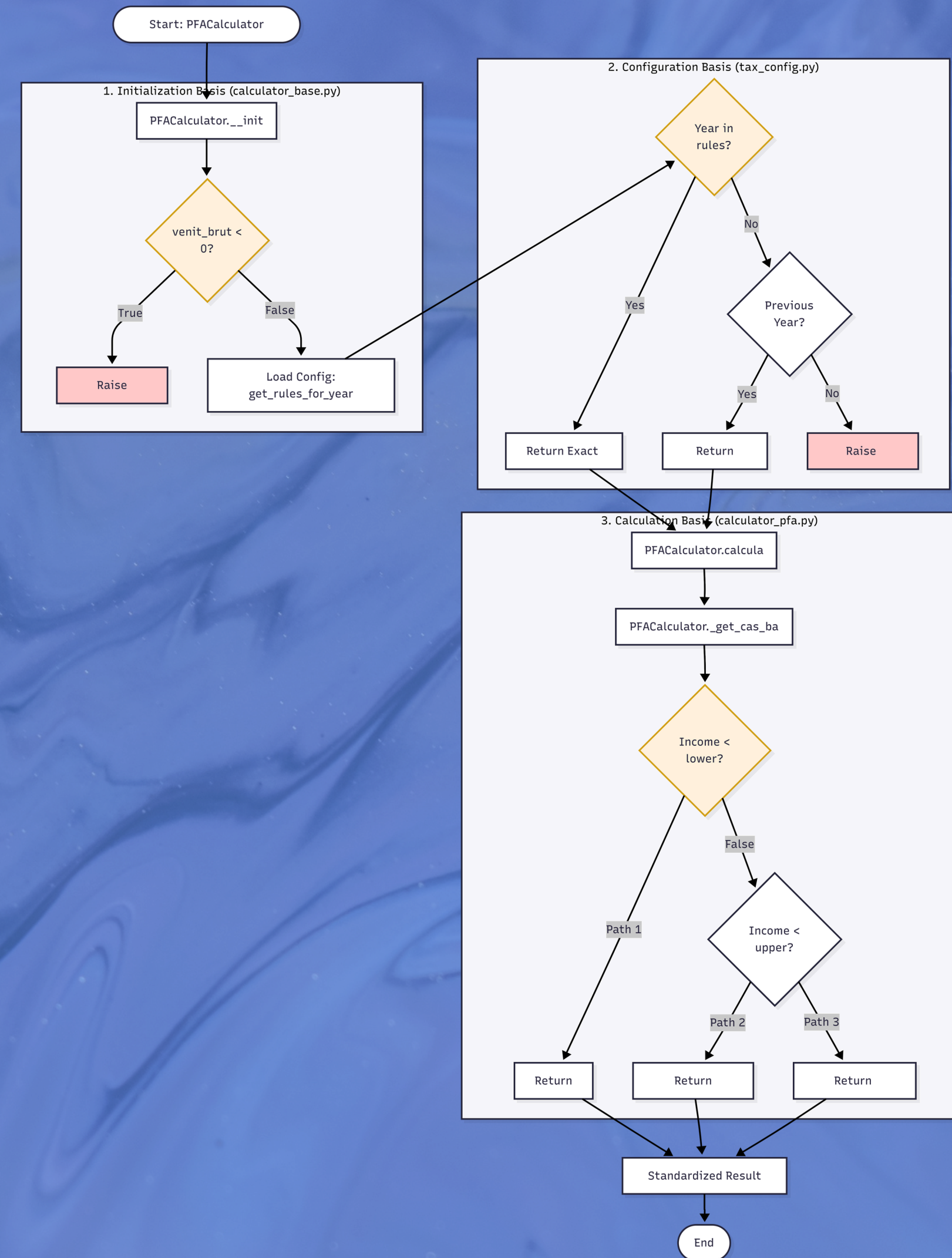
test_statement_coverage.py

Fiecare instrucțiune executată cel puțin o dată.

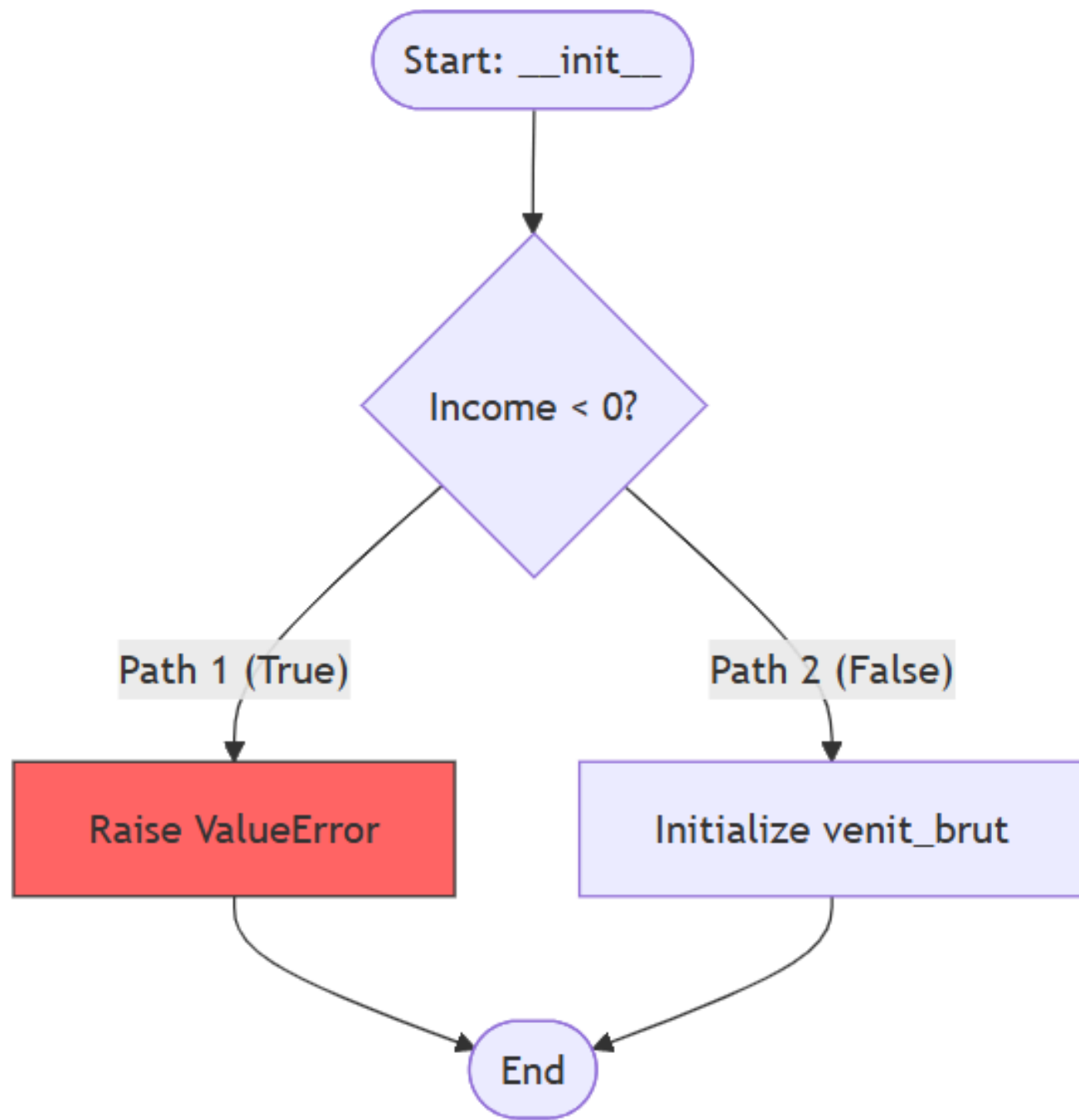
Basis

Test

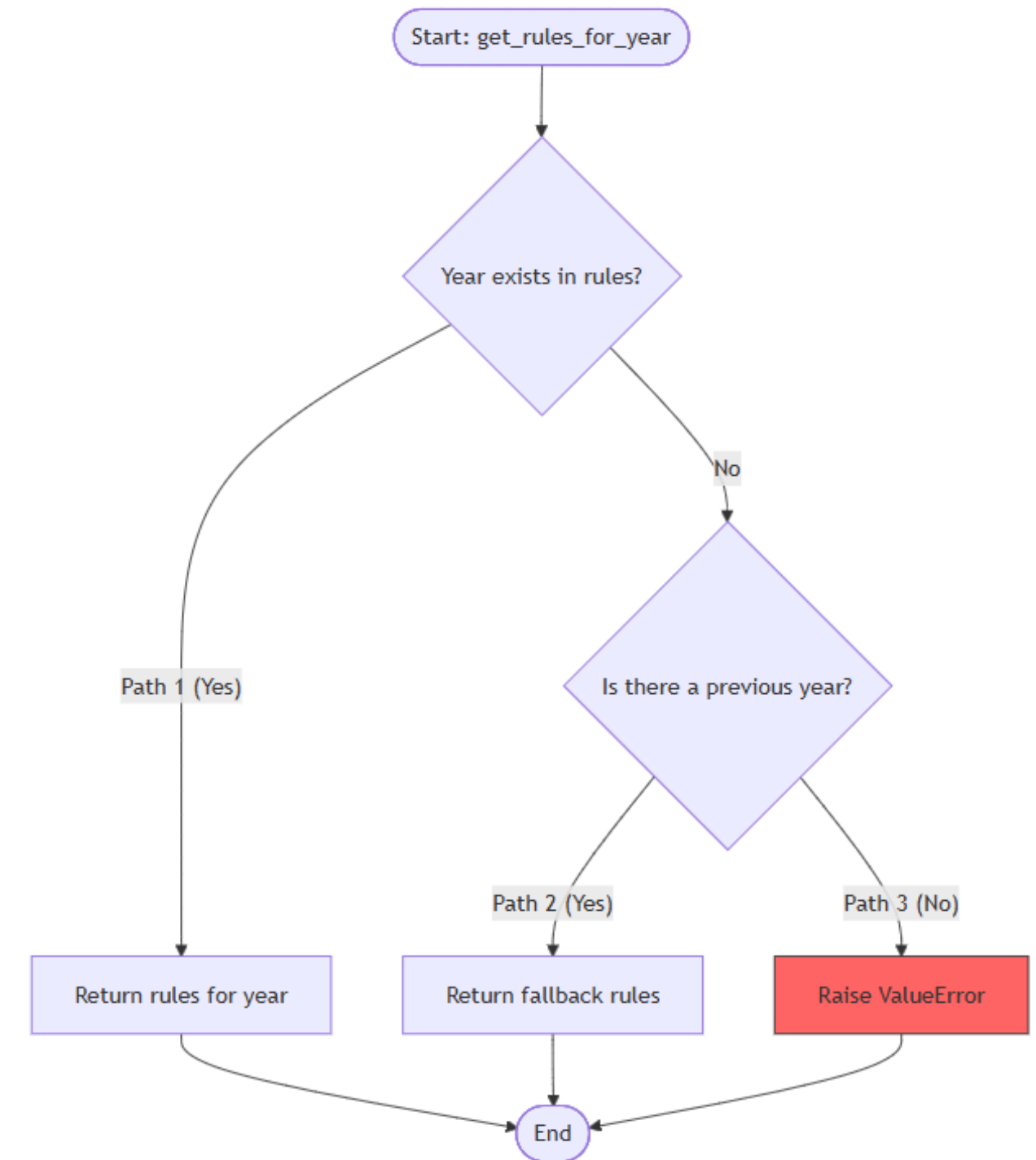
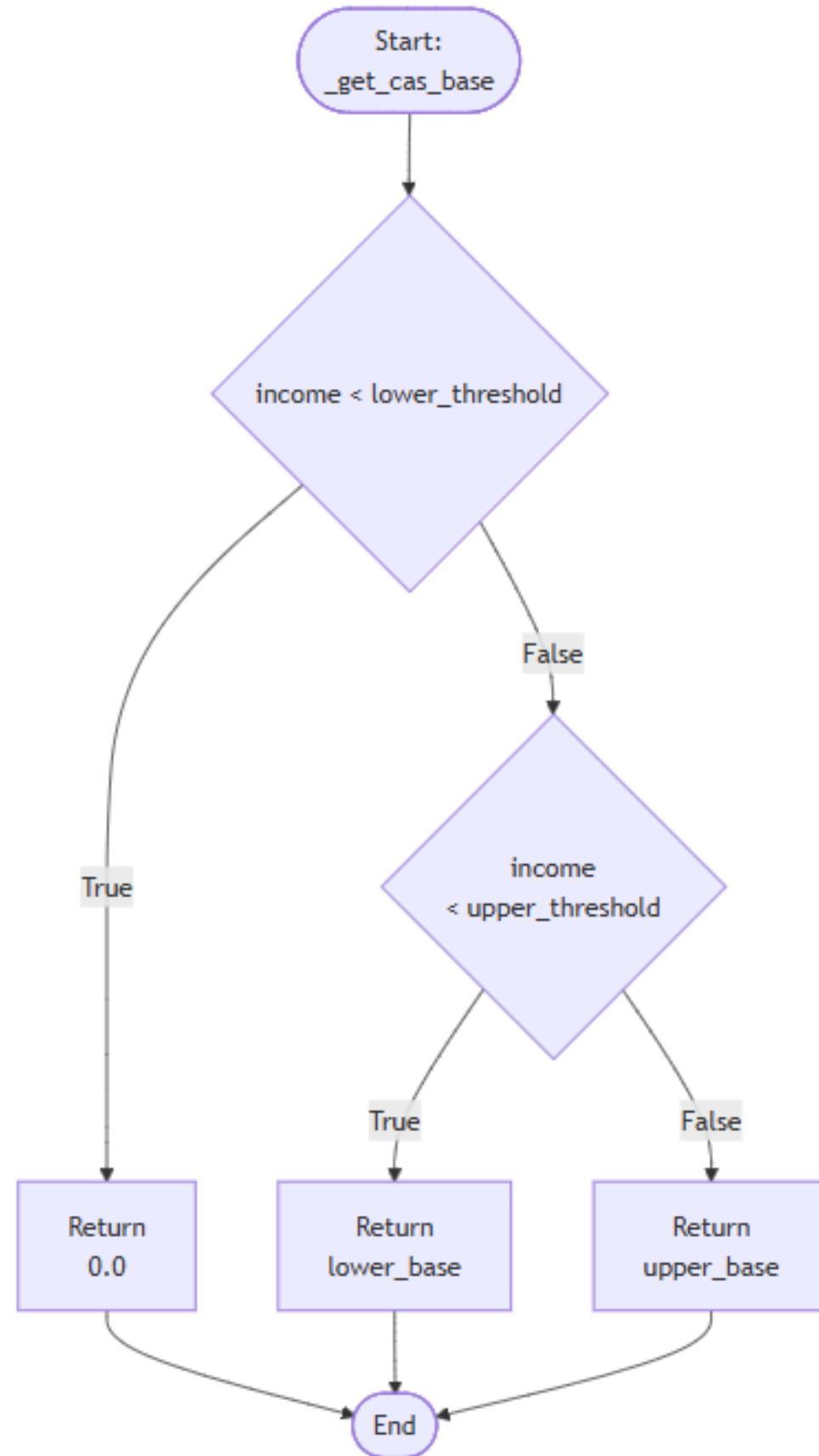
- Testarea verifică toate căile liniar independente, garantând acoperirea fiecărei secvențe unice de execuție.
- Căile sunt deduse prin maparea logicii codului în Grafuri ale Fluxului de Control (CFG). Astfel, fiecare test introduce cel puțin o muchie nouă.
- Complexitatea Ciclomatică: prin analiza nodurilor de decizie din fiecare modul în parte se determină matematic numărul exact de trasee posibile.



CAS Base



Initialization



Tax Config



Testare prin Muta(n)ții

331



Mutanți generați

Concept

Injectarea de defecte artificiale (mutații = mici modificări) în codul sursă (ex: schimbarea `>` în `>=` sau `+` în `-`).

297 → 301



Killed
(89.73% → 90.94%)

Rezultate

Dacă suita de teste eșuează, mutantul este **OMORÂT**. Considerăm asta un succes, fiindcă testele au detectat o neregulă.

34 → 30



Survived
(10.27% → 9.06%)

Rezultate

Dacă testele trec, mutantul a **SUPRAVIEȚUIT**. Acest fapt indică o lacună în teste, o parte de cod pe care acestea nu o acoperă, nu o verifică.

Mutanți Neechivalenți

Aceștia au fost omorâți prin teste suplimentare.

Mutant 1 - Job 23

- **Operator:** ReplaceBinaryOperator_Sub_Add
- **Mutație:** venit_brut - cheltuieli → venit_brut + cheltuieli
- **De ce supraviețuiește:** expense_ratio = 0, deci cheltuieli = 0, operațiile sunt identice
- **Remediu:** monkeypatch expense_ratio = 0.20

Mutant 2 - Job 205

- **Operator:** NumberReplacer (occurrence 2)
- **Mutație:** return 0.0 → return 1.0 în _get_cass_base()
- **De ce supraviețuiește:** linia e dead code - ultimul bracket are max=None
- **Remediu:** monkeypatch cass_brackets = []



Mutanți Echivalenți

De ce nu pot fi omorâți?

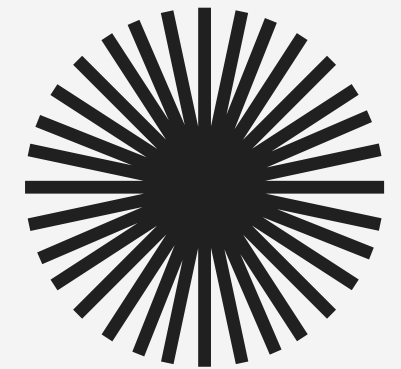
➤ NumberReplacer

Schimbă constante numerice: multiplicatori din brackets (ex. 6 → 7 în max_income_multiplier) și numărul de zecimale din round() (ex. round(cas, 2) → round(cas, 3)), înlocuiesc 2-ul din round(x, 2) pentru fiecare câmp din dicționarul returnat de calculate() cu un alt număr întreg mic .

Supraviețuiesc deoarece valorile din teste sunt numere întregi (ex. 19800.0) - rotunjirea la 2 sau 3 zecimale produce același rezultat. Sunt ECHIVALENȚI în contextul datelor de test curente și deoarece venitul brut și liniile de contribuții din teste se comportă ca numere întregi, deci acele mutații nu schimbă rezultatele observabile.



Mulțumim pentru atenție!



Echipa 18



Dragomir Miruna (G331)

Iștoc Simona (G331)

Tanislav Alexia (G332)

Zamfir Alexandru (G331)